实验报告

课程名称：Design pattern and software architecture

学　　院：计算机科学与工程学院

专　　业:Computer Science and Technology　班　　级：SE-1

姓　　名：Muhammad Samudera　学　　号：

年　　月　　日

山东科技大学教务处制

实 验 报 告

组　别	1	姓　名	Samudera	同组实验者	None
实验项目名称	Bridge Pattern				
教师评语	Good understanding of the pattern idea and implementation. The report is clear and written in proper English.				
实验成绩:		指导教师（签名）： 2026 3 23			

1. Objective

The objective of this experiment is to implement the Bridge design pattern using a photobooth camera system example involving CameraAPI (Implementor) and BoothController (Abstraction). The goal of this pattern is to decouple an abstraction from its implementation so that the two can vary independently.

In this example, the program has two hierarchies:

- CameraAPI (Implementor interface): defines the low-level camera operations such as triggering the shutter, setting exposure, and getting a preview image. Concrete implementations include IPadCameraAPI and RicohGRAPI.
- BoothController (Abstraction): defines the high-level photobooth control logic such as taking a photo and starting a session. It holds a reference to a CameraAPI and delegates camera-specific operations to it. ProBoothController is a refined abstraction that adds advanced features like watermarking and timed sessions.

The Client class acts as the consumer that uses a BoothController without needing to know which specific CameraAPI is being used underneath.

2. Principle

The Bridge pattern separates an abstraction from its implementation by placing them into two separate class hierarchies. Instead of using inheritance to bind the abstraction to a specific implementation, the pattern uses composition — the abstraction holds a reference to an implementor

object and delegates work to it.

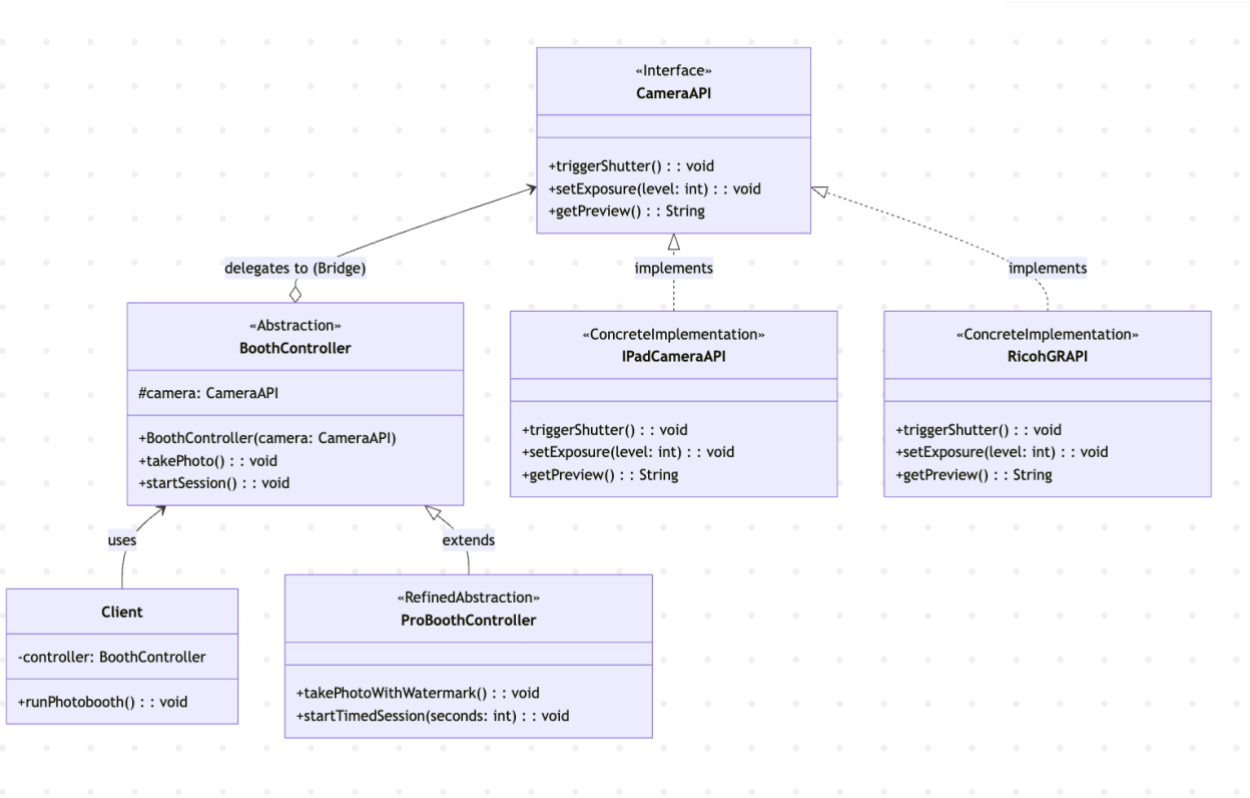
The Bridge pattern separates an abstraction from its implementation by placing them into two separate class hierarchies. Instead of using inheritance to bind the abstraction to a specific implementation, the pattern uses composition — the abstraction holds a reference to an implementor object and delegates work to it.

The Bridge pattern separates an abstraction from its implementation by placing them into two separate class hierarchies. Instead of using inheritance to bind the abstraction to a specific implementation, the pattern uses composition — the abstraction holds a reference to an implementor object and delegates work to it.

Key benefits of this separation include:

- The abstraction (BoothController) and the implementation (CameraAPI) can evolve independently without affecting each other.
- New camera types (e.g., SonyCameraAPI) can be added without modifying any existing booth controller code.
- New booth controller variants (e.g., EventBoothController) can be added without modifying any existing camera implementation.
- The client code is simplified because it only interacts with the abstraction layer, remaining unaware of the concrete implementation details.
- Code duplication is reduced because the same camera implementation can be reused across multiple booth controller types.

3. UML Diagram



4. Key Code (Java)

```
interface CameraAPI {                                    // Implementor

    void triggerShutter();

    void setExposure(int level);

    String getPreview();

}



class IPadCameraAPI implements CameraAPI {                // Concrete Implementor A

    public void triggerShutter() {

        System.out.println("[IPadCameraAPI] Shutter triggered via iPad camera.");

    }
```

```
    public void setExposure(int level) {
        System.out.println("[IPadCameraAPI] Exposure set to level: " + level);
    }
    public String getPreview() {
        return "[IPadCameraAPI] Preview image from iPad camera.";
    }
}

class RicohGRAPI implements CameraAPI {                    // Concrete Implementor B
    public void triggerShutter() {
        System.out.println("[RicohGRAPI] Shutter triggered via Ricoh GR camera.");
    }
    public void setExposure(int level) {
        System.out.println("[RicohGRAPI] Exposure set to level: " + level);
    }
    public String getPreview() {
        return "[RicohGRAPI] Preview image from Ricoh GR camera.";
    }
}

class BoothController {                                    // Abstraction
    protected CameraAPI camera;                            // Bridge reference
```

```
    public BoothController(CameraAPI camera) {
        this.camera = camera;
    }

    public void takePhoto() {
        camera.setExposure(5);
        camera.triggerShutter();
        System.out.println("Preview: " + camera.getPreview());
    }
}

class ProBoothController extends BoothController {          // Refined Abstraction
    public ProBoothController(CameraAPI camera) {
        super(camera);
    }

    public void takePhotoWithWatermark() {
        camera.setExposure(7);
        camera.triggerShutter();
        System.out.println("Preview: " + camera.getPreview());
        System.out.println("Watermark applied.");
```

```
    }


    public void startTimedSession(int seconds) {
        System.out.println("Timed session started for " + seconds + " seconds.");
    }
}


// Client usage — switching implementation without changing abstraction
CameraAPI ipad = new IPadCameraAPI();
BoothController basicBooth = new BoothController(ipad);
basicBooth.takePhoto();      // uses iPad camera


CameraAPI ricoh = new RicohGRAPI();
BoothController ricohBooth = new BoothController(ricoh);
ricohBooth.takePhoto();      // uses Ricoh GR camera — same abstraction, different implementation


ProBoothController proBooth = new ProBoothController(new IPadCameraAPI());
proBooth.takePhotoWithWatermark();      // refined abstraction with iPad camera
```

5. Analysis

- **Advantages:** The Bridge pattern provides a clean separation between the photobooth control logic (abstraction) and the camera hardware operations (implementation). In this program, BoothController does not need to know whether it is working with an iPad camera or a Ricoh GR camera — it simply delegates to the CameraAPI interface. This makes it possible to swap cameras at runtime simply by passing a different CameraAPI instance to the constructor. Furthermore, extending the system is straightforward: adding a new camera type only requires implementing the CameraAPI interface, while adding new booth features only

requires subclassing BoothController. Neither change affects the other hierarchy, adhering to the Open/Closed Principle.

- **Disadvantage:** The Bridge pattern increases the number of classes in the system, which can add complexity to smaller projects where the abstraction and implementation are unlikely to change independently. The indirection introduced by the bridge reference (the CameraAPI field in BoothController) can also make the code harder to trace and debug, since method calls are delegated across two separate hierarchies. Additionally, designing the correct split between the abstraction and implementation interfaces requires careful upfront analysis, and an incorrect split may lead to awkward APIs or unnecessary coupling between the two hierarchies.

6. Conclusion

The Bridge pattern is well-suited for situations where both the abstraction and its implementation need to vary independently, such as in this photobooth camera system. In this program, BoothController defines the high-level session and photo-taking logic, while CameraAPI encapsulates the hardware-specific camera operations. By composing a CameraAPI reference inside the abstraction rather than inheriting from a specific camera class, the system achieves maximum flexibility.

This approach is demonstrated directly in the program output: the same BoothController abstraction produces different behavior when paired with IPadCameraAPI versus RicohGRAPI. Similarly, the ProBoothController refined abstraction adds watermark and timed session capabilities without modifying any camera implementation. The Client class interacts solely with the BoothController abstraction, remaining completely decoupled from the underlying camera details.

Overall, the Bridge pattern is a practical and scalable solution for managing systems where multiple dimensions of variation exist. It reduces the combinatorial explosion of classes that would result from using inheritance alone, promotes code reuse across both hierarchies, and keeps the system maintainable as new abstractions or implementations are added. When the abstraction and implementation are expected to evolve independently, the Bridge pattern serves as an essential structural building block in software architecture.